

Q3 cutoffs under partial missingness

Null simulation of the resample DGP before and after the missingness fix

Magnus Johansson

2026-10-01

Table of contents

Purpose	1
Design	1
Data generation	3
The old resample DGP	3
One dataset	4
Run	5
Results	6
Per-pair rejection	6
Spread and location of the null	7
Family-wise rates	7
Distribution of p-values	8
Both tails and the shift in null SD units	9
Interpretation	10

Purpose

`RMlocdepQ3()` computes the observed Q_3 as a pairwise-complete correlation of residuals, so with partial missingness each item pair rests on the respondents who answered both items. Up to the development version after 1.3.1, `RMlocdepQ3Cutoff()` with the default `dgp = "resample"` simulated **complete** data for every respondent. Each simulated pair therefore had more respondents than the observed pair, and the null distribution was too narrow.

The fix makes each resampled respondent keep their missingness pattern. This study measures what the old behaviour cost and whether the fix restores the nominal rate.

Two things should differ between the arms:

1. **Spread.** The SD of a correlation grows roughly with $1/\sqrt{n}$, so at 30 percent missingness per cell (about half of each pair's respondents left) the observed Q_3 varies about 1.4 times as much as the old null assumed. As a rough estimate, a one-sided 5 percent test then rejects about 13 percent of null pairs.
2. **Location.** Respondents with missing items get WLE estimates from fewer items, so their residuals carry a stronger negative part-whole bias. The old null did not reproduce that shift in the mean Q_3 .

Design

- **Items.** 9 polytomous items with four categories, the item set from the item-restscore studies. $\theta \sim N(0, 1.5^2)$, $n = 300$.
- **Missingness patterns.**
 - `none`: complete data, the reference. The old and new resample arms are identical here by construction, so only the new arm is run.
 - `mcar10`, `mcar30`: each cell missing completely at random with probability .10 or .30.

- ▶ **item50**: item I5 missing for a random half of the respondents, all other items complete. Only the 8 pairs involving I5 are affected, which tests whether the fix works per pair and not just on average.
- **Arms**.
 - ▶ **old**: the resample DGP without the missingness mask, emulated from the package internals with the same seeds as **new**, so the comparison is paired.
 - ▶ **new**: `RMlocdepQ3Cutoff(dgp = "resample")` as it now ships.
 - ▶ **conditional**: `RMlocdepQ3Cutoff(dgp = "conditional")`, which always kept the observed missingness pattern.
- **Iterations**. $B = 200$ per dataset. The marginal per-pair p-value is the main outcome, and it is well resolved at 200 (smallest value 1/201).
- **Replications**. 200 datasets per pattern.
- **Outcomes**.
 - ▶ Per-pair marginal rejection, one-sided upper at $\alpha = .05$, as `RMlocdepQ3()` computes it. Nominal 5 percent.
 - ▶ The same for the I5 pairs and the other pairs separately under **item50**.
 - ▶ Family-wise rate with the Westfall-Young step-down over the 36 pairs.
 - ▶ The global rule, observed max minus mean Q_3 above the 95th percentile of its simulated distribution.

Cost. About 0.3 s per iteration at $n = 300$ (CML), so 60 s per dataset and arm. 10 pattern-arm combinations $\times$ 200 datasets is about 35 CPU-hours, or 3.5 to 4 hours on 10 cores. Results are saved per pattern and arm, so an interrupted render resumes.

```
suppressMessages({
  devtools::load_all("...", quiet = TRUE)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})
options(rgl.useNULL = TRUE)

R_DATASETS <- 200
B          <- 200
N          <- 300
THETA_SD   <- 1.5
NCORES     <- 10
RES_DIR    <- "q3_missingness_null"
dir.create(RES_DIR, showWarnings = FALSE)

thr <- lapply(seq(-1.5, 1.5, length.out = 9), function(l) l + c(-1.2, 0, 1.2))
item_names <- paste0("I", seq_along(thr))

design <- data.frame(
  pattern = c("none", rep(c("mcar10", "mcar30", "item50"), each = 3)),
  arm     = c("new", rep(c("old", "new", "conditional"), 3)),
  stringsAsFactors = FALSE
)
design
```

pattern	arm
none	new
mcar10	old

pattern	arm
mcar10	new
mcar10	conditional
mcar30	old
mcar30	new
mcar30	conditional
item50	old
item50	new
item50	conditional

Data generation

```

sim_pcm <- function(theta, thr) {
  n <- length(theta)
  sapply(thr, function(d) {
    m <- length(d)
    eta <- outer(theta, 0:m) -
      matrix(c(0, cumsum(d)), n, m + 1, byrow = TRUE)
    p <- exp(eta - apply(eta, 1, max))
    p <- p / rowSums(p)
    cp <- p %*% upper.tri(diag(m + 1), diag = TRUE)
    rowSums(runif(n) > cp)
  })
}

# Complete data with every category observed, then the missingness pattern.
# The dataset depends only on (pattern, rep), so all arms see the same data.
make_data <- function(pattern, rep) {
  set.seed(1e5 * match(pattern, unique(design$pattern)) + rep)
  repeat {
    d <- sim_pcm(rnorm(N, 0, THETA_SD), thr)
    if (all(apply(d, 2, function(x) all(tabulate(x + 1, 4) > 0)))) break
  }
  colnames(d) <- item_names
  if (pattern %in% c("mcar10", "mcar30")) {
    rate <- if (pattern == "mcar10") 0.10 else 0.30
    d[matrix(runif(length(d)) < rate, nrow(d))] <- NA
  } else if (pattern == "item50") {
    d[sample(N, N / 2), "I5"] <- NA
  }
  d <- d[rowSums(!is.na(d)) > 0, , drop = FALSE]
  as.data.frame(d)
}

```

The old resample DGP

This rebuilds the simulation list the way `RMlocdepQ3Cutoff()` does, with the same per-iteration seeds, but without the missingness mask. It reproduces the pre-fix code path, which generated complete data.

```

old_cutoff <- function(d, B, seed) {
  dm <- as.matrix(d)

```

```

pool <- .wle_theta_pool(dm)
dl <- list(
  dgp = "resample",
  type = "polytomous",
  thr_list = pool$thr_list,
  n_items = ncol(dm),
  sample_n = nrow(dm),
  item_names = colnames(dm),
  estimator = "CML",
  thetas = pool$thetas,
  deltaslist = pool$thr_list
)
set.seed(seed)
seeds <- sample.int(.Machine$integer.max, B)
raw <- lapply(seeds, run_single_q3_sim, data_list = dl)
ok <- Filter(is.list, raw)
list(
  results = data.frame(
    mean = vapply(ok, `[`, numeric(1), "mean"),
    max = vapply(ok, `[`, numeric(1), "max")
  ),
  pair_results = do.call(rbind, lapply(seq_along(ok), function(i) {
    cbind(ok[[i]]$pair_q3, iteration = i)
  })),
  actual_iterations = length(ok)
)
}

```

The emulation is checked against the package on complete data, where the two must agree exactly.

```

d0 <- make_data("none", 1)
pkg <- suppressMessages(RMlocdepQ3Cutoff(d0, iterations = 5, parallel = FALSE,
  seed = 11))

emu <- old_cutoff(d0, 5, 11)
stopifnot(isTRUE(all.equal(pkg$pair_results$Q3, emu$pair_results$Q3)))
"old-arm emulation matches the package on complete data"

```

```
[1] "old-arm emulation matches the package on complete data"
```

One dataset

The p-values are computed the way `RMlocdepQ3()` computes them: the observed pairwise-complete Q_3 against each pair's simulated distribution, one-sided upper, studentised by the bootstrap mean and SD.

```

one_dataset <- function(rep, pattern, arm) {
  d <- make_data(pattern, rep)
  seed <- 1e5 * match(pattern, unique(design$pattern)) + rep
  co <- tryCatch({
    if (arm == "old") {
      old_cutoff(d, B, seed)
    } else {
      suppressMessages(RMlocdepQ3Cutoff(
        d, iterations = B, parallel = FALSE, seed = seed,
        cutoff_method = "quantile", dgp = if (arm == "new") "resample" else arm
      ))
    }
  })
}

```

```

    ))
  }
}, error = function(e) NULL)
if (is.null(co)) return(NULL)

obs <- .q3_residual_matrix(d, estimator = "CML")
pr <- co$pair_results
pr$key <- paste(pr$Item1, pr$Item2, sep = "__")
sim_mat <- tapply(pr$Q3, list(pr$iteration, pr$key), function(x) x[1L])
pairs <- unique(pr[, c("Item1", "Item2", "key")])
observed <- stats::setNames(
  mapply(function(a, b) obs[a, b], pairs$Item1, pairs$Item2),
  pairs$key
)
p_none <- .bootstrap_pvalues(observed, sim_mat, "none", tail = "upper")
p_wy <- .bootstrap_pvalues(observed, sim_mat, "fwer", tail = "upper")

obs_diff <- max(obs, na.rm = TRUE) - mean(obs, na.rm = TRUE)
sim_diff <- co$results$max - co$results$mean

data.frame(
  rep = rep,
  Item1 = pairs$Item1,
  Item2 = pairs$Item2,
  observed = as.numeric(observed),
  null_mean = colMeans(sim_mat, na.rm = TRUE)[pairs$key],
  null_sd = apply(sim_mat, 2, stats::sd, na.rm = TRUE)[pairs$key],
  p = p_none$p[match(pairs$key, p_none$name)],
  padj_wy = p_wy$padj[match(pairs$key, p_wy$name)],
  global_flag = obs_diff > stats::quantile(sim_diff, 0.95),
  iterations = co$actual_iterations,
  n = nrow(d),
  stringsAsFactors = FALSE,
  row.names = NULL
)
}

```

Run

```

for (i in seq_len(nrow(design))) {
  f <- file.path(RES_DIR, paste0(design$pattern[i], "_", design$arm[i], ".rds"))
  if (file.exists(f)) next
  t0 <- Sys.time()
  res <- mclapply(seq_len(R_DATASETS),
    function(r) one_dataset(r, design$pattern[i], design$arm[i]),
    mc.cores = NCORES, mc.preschedule = FALSE)
  saveRDS(list(res = res,
    elapsed_min = as.numeric(Sys.time() - t0, units = "mins")),
    f)
}

long <- do.call(rbind, lapply(seq_len(nrow(design)), function(i) {
  f <- file.path(RES_DIR, paste0(design$pattern[i], "_", design$arm[i], ".rds"))
  x <- readRDS(f)

```

```

ok <- Filter(Negate(is.null), x$res)
cbind(pattern = design$pattern[i], arm = design$arm[i],
       do.call(rbind, ok), elapsed_min = x$elapsed_min,
       failed = length(x$res) - length(ok))
}))
long$pattern <- factor(long$pattern, levels = unique(design$pattern))
long$arm <- factor(long$arm, levels = c("old", "new", "conditional"))
long$i5_pair <- long$item1 == "I5" | long$item2 == "I5"

long |>
  group_by(pattern, arm) |>
  summarise(datasets = n_distinct(rep),
            failed = first(failed),
            median_iterations = median(iterations[!duplicated(rep)]),
            minutes = round(first(elapsed_min), 1),
            .groups = "drop")

```

pattern	arm	datasets	failed	median_iterations	minutes
none	new	200	0	200	2.6
mcar10	old	200	0	200	2.9
mcar10	new	200	0	200	21.7
mcar10	conditional	200	0	200	30.2
mcar30	old	200	0	200	3.3
mcar30	new	200	0	200	51.8
mcar30	conditional	200	0	200	71.4
item50	old	200	0	200	2.7
item50	new	200	0	200	3.8
item50	conditional	200	0	200	5.5

Results

Per-pair rejection

Marginal one-sided p-value below .05. Nominal 5 percent.

```

long |>
  group_by(pattern, arm) |>
  summarise(reject = round(100 * mean(p < .05), 2), .groups = "drop") |>
  pivot_wider(names_from = arm, values_from = reject)

```

pattern	new	old	conditional
none	4.51	NA	NA
mcar10	4.93	4.69	5.29
mcar30	4.57	4.33	4.94
item50	4.86	5.35	4.86

Under `item50`, the pairs involving I5 and the other pairs separately:

```
long |>
  filter(pattern == "item50") |>
  mutate(pairs = ifelse(i5_pair, "with I5", "without I5")) |>
  group_by(pairs, arm) |>
  summarise(reject = round(100 * mean(p < .05), 2), .groups = "drop") |>
  pivot_wider(names_from = arm, values_from = reject)
```

pairs	old	new	conditional
with I5	12.25	5.19	4.94
without I5	3.38	4.77	4.84

Spread and location of the null

`sd_ratio` is the empirical SD of the observed Q_3 across datasets divided by the average simulated SD, per pair, averaged over pairs. `mean_gap` is the average observed Q_3 minus the simulated mean. A calibrated null gives 1 and 0.

```
long |>
  group_by(pattern, arm, Item1, Item2) |>
  summarise(sd_ratio = sd(observed) / mean(null_sd),
            mean_gap = mean(observed - null_mean),
            .groups = "drop") |>
  group_by(pattern, arm) |>
  summarise(sd_ratio = round(mean(sd_ratio), 3),
            mean_gap = round(mean(mean_gap), 4),
            .groups = "drop")
```

pattern	arm	sd_ratio	mean_gap
none	new	0.989	-0.0006
mcar10	old	1.114	-0.0127
mcar10	new	1.003	-0.0008
mcar10	conditional	1.007	-0.0001
mcar30	old	1.420	-0.0458
mcar30	new	0.996	-0.0014
mcar30	conditional	1.003	-0.0003
item50	old	1.079	-0.0073
item50	new	0.988	-0.0008
item50	conditional	0.990	-0.0002

Family-wise rates

At least one pair flagged by the Westfall-Young step-down, and the global max minus mean rule at its 95th percentile. Both nominal 5 percent. The Monte Carlo SE at 200 datasets is 1.5 points.

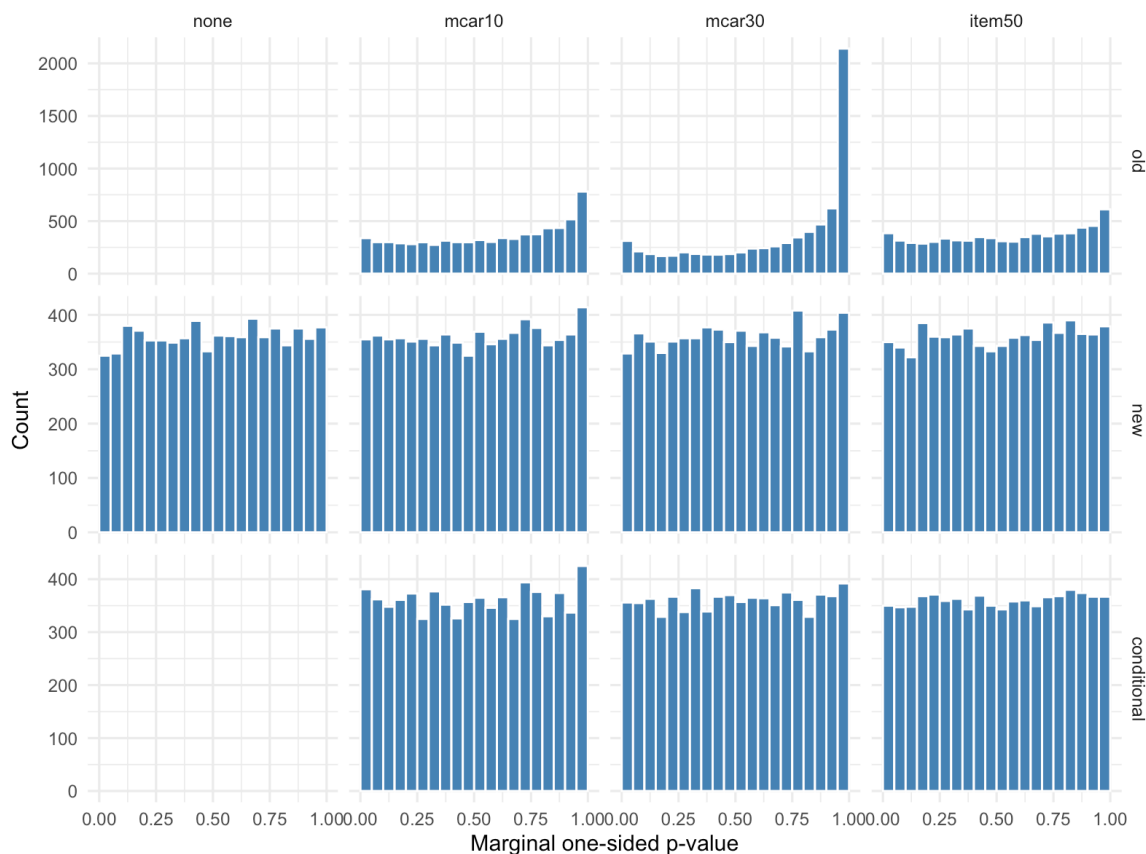
```
long |>
  group_by(pattern, arm, rep) |>
  summarise(wy = any(padj_wy < .05), global = first(global_flag),
            .groups = "drop") |>
  group_by(pattern, arm) |>
```

```
summarise(wy = round(100 * mean(wy), 1),
          global = round(100 * mean(global), 1),
          .groups = "drop")
```

pattern	arm	wy	global
none	new	6.0	6.5
mcar10	old	8.5	15.0
mcar10	new	5.5	5.0
mcar10	conditional	6.5	4.0
mcar30	old	16.5	47.0
mcar30	new	5.5	5.0
mcar30	conditional	4.0	5.5
item50	old	17.0	15.5
item50	new	7.0	3.5
item50	conditional	6.0	4.0

Distribution of p-values

```
long |>
  ggplot(aes(p)) +
  geom_histogram(breaks = seq(0, 1, 0.05), fill = "steelblue",
                 colour = "white") +
  facet_grid(arm ~ pattern, scales = "free_y") +
  labs(x = "Marginal one-sided p-value", y = "Count") +
  theme_minimal()
```



Both tails and the shift in null SD units

The per-pair rejection above uses the upper tail only. `upper_05` repeats it, `lower_05` is the share of p-values above .95 (the lower tail), and `gap_sd` is the average of (observed minus simulated mean) divided by the simulated SD, so the location error is on the same scale as the test.

```
long |>
  group_by(pattern, arm) |>
  summarise(upper_05 = round(100 * mean(p < .05), 2),
            lower_05 = round(100 * mean(p > .95), 2),
            gap_sd = round(mean((observed - null_mean) / null_sd), 3),
            .groups = "drop")
```

pattern	arm	upper_05	lower_05	gap_sd
none	new	4.51	5.24	-0.011
mcar10	old	4.69	10.85	-0.228
mcar10	new	4.93	5.75	-0.012
mcar10	conditional	5.29	5.90	-0.001
mcar30	old	4.33	29.74	-0.824
mcar30	new	4.57	5.61	-0.019
mcar30	conditional	4.94	5.44	-0.004
item50	old	5.35	8.50	-0.129
item50	new	4.86	5.26	-0.011
item50	conditional	4.86	5.10	-0.002

```

long |>
  filter(pattern == "item50") |>
  mutate(pairs = ifelse(i5_pair, "with I5", "without I5")) |>
  group_by(arm, pairs, Item1, Item2) |>
  summarise(sd_ratio = sd(observed) / mean(null_sd),
            gap_sd = mean((observed - null_mean) / null_sd),
            .groups = "drop") |>
  group_by(arm, pairs) |>
  summarise(sd_ratio = round(mean(sd_ratio), 3),
            gap_sd = round(mean(gap_sd), 3), .groups = "drop")

```

arm	pairs	sd_ratio	gap_sd
old	with I5	1.412	-0.010
old	without I5	0.983	-0.163
new	with I5	0.993	-0.014
new	without I5	0.986	-0.011
conditional	with I5	0.997	-0.008
conditional	without I5	0.987	0.000

Interpretation

1. The fix restores the nominal rate. With the missingness mask, the resample DGP rejected 4.6 to 4.9 percent of null pairs at $\alpha = .05$ under all three missingness patterns, against 4.5 percent with complete data. The simulated null has the right spread (SD ratio 0.99 to 1.00) and the right centre (mean gap below 0.02 null SDs). The Westfall-Young family-wise rate was 5.5 to 7.0 percent and the global max-minus-mean rule 3.5 to 5.0 percent, within Monte Carlo error of 5 percent (SE 1.5 points at 200 datasets). The conditional DGP, which always kept the observed missingness, gives the same picture.

2. The old null was wrong in two ways that cancel in the per-pair test. At 30 percent MCAR the old null was too narrow (SD ratio 1.42) and centred too high (the observed Q_3 fell 0.82 null SDs below the simulated mean on average). The first error pushes the upper-tail rejection up and the second pushes it down, and at $\alpha = .05$ they cancelled to 4.3 percent. The lower tail shows what was hidden: 29.7 percent of p-values fell above .95. The family-wise rules do not cancel, because the maximum over 36 pairs is driven by spread and the max-minus-mean statistic largely removes the shift. Under the old null, WY flagged 16.5 percent of datasets and the global rule 47.0 percent. At 10 percent MCAR the same pattern is weaker (WY 8.5, global 15.0 percent).

3. The shift comes from the respondents with missing items. Under `item50` the two errors separate. Pairs involving I5 got the narrow null (SD ratio 1.41) without a shift and rejected 12.25 percent, close to the 13 percent predicted in the Purpose section from spread alone. The other pairs got the shift without the extra spread (-0.16 null SDs) and rejected 3.4 percent. Respondents who skipped I5 have WLE estimates based on eight items, so their residuals on the other items carry a stronger negative part-whole bias, which lowers Q_3 for pairs that do not include I5. The old null, built on complete data, had neither feature.

4. The fix is slower with many missingness patterns. For the same 200 datasets and 200 iterations, the new resample arm took 21.7 minutes at 10 percent MCAR and 51.8 at 30 percent, against 2.9 and 3.3 for the old arm. Under `item50`, with only two patterns, it took 3.8 minutes against 2.7. The cause has not been measured. The likely one is the CML refit, which is fast on complete data and slows with the number of distinct missingness patterns.

Implications. The bug fix in NEWS is warranted, and the size of the old error (a global rule rejecting in nearly half of null datasets at 30 percent MCAR) is worth stating in the entry. Users with substantial MCAR missingness will see a several-fold longer run time, which may need a sentence in the help page.